

Attention Mechanism — Encoder-Decoder

Detailed notes based on our discussion, including dimensions, Bahdanau attention, numerical calculations, Softmax, attention weights, and the context vector.

1. Architecture we will use throughout

To avoid dimension confusion, keep this exact architecture fixed:

Part	Configuration	Dimension
Encoder	Bidirectional LSTM: 256 forward + 256 backward	$H_1, H_2, H_3 = 512$
Encoder final hidden	Forward + backward concatenated	512
Encoder final cell	Forward + backward concatenated	512
Decoder	Normal LSTM	256 units
Decoder initial hidden S_0	Projection from encoder final hidden	256
Decoder initial cell C_0	Projection from encoder final cell	256
Attention space	Learned projections	128

Key rule: 256 units in each BiLSTM direction means the concatenated output is $256 + 256 = 512$. It is not 256.

2. Encoder outputs and final states

For input: I → love → mangoes, the BiLSTM produces one output per position:

$H_1 = [\text{forward } 256 ; \text{backward } 256] = 512\text{-D}$

$H_2 = [\text{forward } 256 ; \text{backward } 256] = 512\text{-D}$

$H_3 = [\text{forward } 256 ; \text{backward } 256] = 512\text{-D}$

The final hidden and cell states are also formed from the two directions:

Encoder final hidden = $256 + 256 = 512$

Encoder final cell = $256 + 256 = 512$

3. How S_0 and C_0 are obtained

Our decoder has 256 units, so its hidden and cell states must be 256-D. The encoder states are 512-D. Therefore they are projected separately:

Encoder final hidden [512] → Dense/Linear → S_0 [256]

Encoder final cell [512] → Dense/Linear → C_0 [256]

Do not concatenate H and C into a 1024-D vector. Hidden state and cell state are different LSTM states and are projected separately.

4. Decoder starts

During training, the first decoder input is SOS (start of sequence). If embedding size is 128:

SOS → Embedding → X_1 [128]

Decoder LSTM receives X_1 [128], S_0 [256], C_0 [256] and produces S_1 [256], C_1 [256].

Attention is then used to determine which encoder positions are important for the current decoding step.

5. Bahdanau attention

For decoder step 1, compare S_0 with every encoder output H_1, H_2, H_3 . Because S_0 is 256-D and H_i is 512-D, they are first projected into a common attention space of 128 dimensions.

$W_s S_0 \rightarrow 128\text{-D}$

$W_h H_i \rightarrow 128\text{-D}$

$E_{1,i} = \tanh(W_s S_0 + W_h H_i + b) \rightarrow 128\text{-D vector}$

$e_{1,i} = v_a^T E_{1,i} \rightarrow \text{scalar alignment score}$

v_a is a learned parameter vector. It is not three separate neural networks. The same scoring parameters are shared for H_1, H_2, H_3 .

6. Numerical example — calculating $E_{1,1}$, $E_{1,2}$, $E_{1,3}$

To show every arithmetic operation, use 3-D toy vectors. The real model uses 256/512/128 dimensions, but the calculation is identical.

$$S_0 = [0.5, 0.2, 0.1]$$

$$H_1 = [0.8, 0.1, 0.4]$$

$$H_2 = [0.2, 0.7, 0.3]$$

$$H_3 = [0.1, 0.3, 0.9]$$

For this toy example only, let W_s and W_h be identity matrices and ignore the bias. Thus $E_{1,i} = \tanh(S_0 + H_i)$.

$E_{1,1}$

$$S_0 + H_1 = [0.5, 0.2, 0.1] + [0.8, 0.1, 0.4] = [1.3, 0.3, 0.5]$$

$$\tanh \rightarrow E_{1,1} = [0.862, 0.291, 0.462]$$

$E_{1,2}$

$$S_0 + H_2 = [0.5, 0.2, 0.1] + [0.2, 0.7, 0.3] = [0.7, 0.9, 0.4]$$

$$\tanh \rightarrow E_{1,2} = [0.604, 0.716, 0.380]$$

$E_{1,3}$

$$S_0 + H_3 = [0.5, 0.2, 0.1] + [0.1, 0.3, 0.9] = [0.6, 0.5, 1.0]$$

$$\tanh \rightarrow E_{1,3} = [0.537, 0.462, 0.762]$$

Important: $E_{1,1}$, $E_{1,2}$ and $E_{1,3}$ are still vectors. They are not attention probabilities yet.

7. Convert E vectors into scalar alignment scores

Let the learned scoring vector be $v_a = [0.5, 0.3, 0.2]$.

$$e_{1,i} = v_a^T E_{1,i}$$

$$e_{1,1} = (0.5)(0.862) + (0.3)(0.291) + (0.2)(0.462) = 0.6107$$

$$e_{1,2} = (0.5)(0.604) + (0.3)(0.716) + (0.2)(0.380) = 0.5928$$

$$e_{1,3} = (0.5)(0.537) + (0.3)(0.462) + (0.2)(0.762) = 0.5595$$

Raw scores = [0.6107, 0.5928, 0.5595]

8. Softmax → attention weights $A_{1,1}$, $A_{1,2}$, $A_{1,3}$

Softmax is applied across all three scalar scores for this decoder step:

$$A_{1,i} = \exp(e_{1,i}) / [\exp(e_{1,1}) + \exp(e_{1,2}) + \exp(e_{1,3})]$$

$$\exp(0.6107) \approx 1.842; \exp(0.5928) \approx 1.809; \exp(0.5595) \approx 1.749$$

$$\text{Denominator} \approx 1.842 + 1.809 + 1.749 = 5.400$$

$$A_{1,1} = 1.842 / 5.400 \approx 0.341$$

$$A_{1,2} = 1.809 / 5.400 \approx 0.335$$

$$A_{1,3} = 1.749 / 5.400 \approx 0.324$$

$A_1 = [0.341, 0.335, 0.324]$; the weights sum to 1.

Do not take only the maximum. Attention uses all three weights. The weights tell us how strongly each encoder vector contributes to the context.

9. Weighted sum → context vector

A is scalar; H is a vector. Therefore scalar $\times$ vector produces a vector of the same size.

$$\mathbf{A}_{1,1}\mathbf{H}_1 = 0.341[0.8,0.1,0.4] = [0.2728,0.0341,0.1364]$$

$$\mathbf{A}_{1,2}\mathbf{H}_2 = 0.335[0.2,0.7,0.3] = [0.0670,0.2345,0.1005]$$

$$\mathbf{A}_{1,3}\mathbf{H}_3 = 0.324[0.1,0.3,0.9] = [0.0324,0.0972,0.2916]$$

$$\mathbf{Context}_1 = \mathbf{A}_{1,1}\mathbf{H}_1 + \mathbf{A}_{1,2}\mathbf{H}_2 + \mathbf{A}_{1,3}\mathbf{H}_3$$

$$\mathbf{Context}_1 = [0.3722, 0.3658, 0.5285]$$

In the real model, $\mathbf{H}_1, \mathbf{H}_2, \mathbf{H}_3$ are 512-D, so $\mathbf{Context}_1$ is also 512-D.

10. Complete mental model

At decoder step 1:

H_1, H_2, H_3 [512] + S_0 [256]

→ project into common 128-D attention space

→ $E_{1,1}, E_{1,2}, E_{1,3}$ [128-D vectors]

→ shared scoring with v_a

→ $e_{1,1}, e_{1,2}, e_{1,3}$ [scalars]

→ one Softmax across the three scores

→ $A_{1,1}, A_{1,2}, A_{1,3}$ [scalar weights]

→ $A_{1,1}H_1 + A_{1,2}H_2 + A_{1,3}H_3$

→ Context_1 [512-D]

11. Four objects you must never confuse

Object	Meaning	Example
$E_{1,1}$	Intermediate attention representation	128-D vector
$e_{1,1}$	Raw alignment score	Scalar
$A_{1,1}$	Softmax attention weight	Scalar
H_1	Encoder output at position 1	512-D vector
Context_1	Weighted sum of encoder outputs	512-D vector

12. Final checklist

- BiLSTM 256 per direction → output 512.
- Encoder final hidden and cell states can each be 512 after concatenating directions.
- Decoder 256 units → S_0 and C_0 must be 256.
- Project encoder final hidden and cell states separately: 512 → 256.
- Do not combine H and C into a 1024-D initialization vector.
- E vectors are vectors; alignment scores are scalars.
- v_a is a learned scoring vector shared across positions.
- Softmax is applied across all alignment scores for one decoder timestep.
- Attention weights are not chosen by argmax; all are used.
- The context vector is the weighted sum of all encoder outputs.